

WePayU

Sistema de Gerenciamento de Folha de Pagamento

Documentação Técnica Abrangente da Arquitetura, Classes, Métodos e Regras de
Negócio

1º Milestone (User Stories 1 a 8)

Disciplina: Programação 2 (P2)

Instituição: Universidade Federal de Alagoas (UFAL)

Linguagem: Java (JDK 25 / compatível com Java 17+)

Testes de Aceitação: EasyAccept (100% de Aprovação em 590 Testes)

Data de Emissão: 2026

Sumário

- 1. Visão Geral do Sistema** — Objetivos, funcionalidades e regras gerais
- 2. Arquitetura de Software e Padrões de Projeto** — Separação de camadas, Façade e Memento
- 3. Resumo das User Stories Implementadas (US1 a US8)** — Visão funcional
- 4. Documentação das Classes e Métodos**
 - 4.1. Camada de Fachada: `Facade.java`
 - 4.2. Núcleo de Regras de Negócio: `Sistema.java`
 - 4.3. Motor de Cálculo e Relatórios: `FolhaDePagamento.java`
 - 4.4. Modelo de Domínio: `Empregado` e Especializações
 - 4.5. Registros Auxiliares: `point_card_pay_u` , `ResultadoVenda` , `TaxaServico`
 - 4.6. Tratamento de Exceções: `EmpregadoNaoExisteException`
- 5. Ciclo de Vida de Execução, Undo/Redo e Persistência**
- 6. Guia de Compilação e Execução dos Testes**

1. Visão Geral do Sistema

O **WePayU** é um sistema corporativo completo voltado à gestão e processamento da folha de pagamento de empregados de uma organização. O software provê um conjunto robusto de operações para cadastro de colaboradores, lançamento de jornadas de trabalho, vendas comissionadas, gerenciamento de contribuições sindicais, emissão detalhada de contracheques periódicos e um histórico transacional completo que permite operações de reversão (**Undo**) e reaplicação (**Redo**).

1.1. Categorias de Empregados Atendidas

- **Horistas** (**EmpregadoHorista**): Recebem remuneração calculada estritamente pelas horas trabalhadas, registradas em cartões de ponto diários. A jornada padrão diária é de até 8 horas normais; horas trabalhadas além desse limite são bonificadas com taxa adicional de 50% (fator 1.5x). A folha é rodada semanalmente (toda sexta-feira).
- **Assalariados** (**EmpregadoAssalariado**): Recebem um valor salarial mensal fixo contratual. O pagamento ocorre no último dia útil de cada mês.
- **Comissionados** (**EmpregadoComissionado**): Subtipo de empregado assalariado que recebe um salário base fixo mensal (calculado proporcionalmente às semanas do período) somado a uma comissão percentual aplicada sobre o valor bruto das vendas comprovadas. O pagamento ocorre a cada duas sextas-feiras.

1.2. Canais de Pagamento

Cada empregado define livremente como deseja receber sua remuneração:

- **emMaos** : Pagamento em dinheiro/cheque entregue em mãos (padrão do sistema).
- **correios** : Envio de cheque nominal via correio para o endereço cadastrado.
- **banco** : Depósito bancário direto, exigindo nome do banco, agência e conta corrente.

1.3. Afiliação Sindical e Deduções

Empregados podem optar por filiar-se ao sindicato da categoria. Ao sindicalizar-se, é gerado um identificador sindical exclusivo (independente da matrícula interna na empresa), uma taxa fixa periódica e a possibilidade de cobrança de taxas de serviços adicionais (creche, assistência médica, jurídica, etc.).

No processamento da folha, todas as taxas e débitos sindicais são deduzidos do salário bruto. Caso as deduções superem o salário líquido, o valor excedente é mantido como débito pendente para desconto na próxima folha.

2. Arquitetura de Software e Padrões de Projeto

O sistema foi concebido sob princípios de baixo acoplamento, alta coesão e separação rigorosa entre a interface de comunicação externa e a lógica de domínio interna.

2.1. Padrão Façade (Fachada)

A classe `Facade` atua como ponto único de entrada para todas as operações do sistema. O framework de testes de aceitação (**EasyAccept**) interage exclusivamente com a fachada, sem acessar coleções ou classes de negócio diretamente. A fachada é responsável por delegar as chamadas para a instância ativa de `Sistema`, garantindo uma barreira arquitetural limpa.

2.2. Padrão Memento com Snapshots em Memória (Undo / Redo)

Para atender aos requisitos rigorosos da US8, o sistema adota o padrão **Memento**. Toda e qualquer operação que altere o estado do sistema (criação, remoção, alteração de empregados, lançamentos de cartões, taxas, vendas ou execução da folha) é interceptada:

- Um snapshot completo e isolado do estado do sistema é serializado em memória na forma de array de bytes (`byte[]`).
- A ação modificadora é executada. Se ocorrer sucesso, o snapshot do estado anterior é empilhado na `undoStack` e a `redoStack` é esvaziada.
- Se a ação lançar qualquer exceção (ex: validação de data, valor negativo, etc.), o sistema restaura imediatamente o snapshot anterior (**Rollback**), impedindo estados intermediários corrompidos e sem poluir o histórico.
- Ao invocar `undo()`, o estado atual vai para a `redoStack` e o topo da `undoStack` é restaurado. Ao invocar `redo()`, o processo inverso ocorre.

2.3. Herança e Polimorfismo

A hierarquia de domínio possui `Empregado` como classe abstrata raiz, provendo reuso de atributos comuns (identificação, dados sindicais e bancários) e comportamento polimórfico especializado para `EmpregadoHorista`, `EmpregadoAssalariado` e `EmpregadoComissionado`. Conforme exigido pela especificação oficial do projeto, o uso consciente de `instanceof` permite discriminar o tipo de contrato sem violar o encapsulamento.

3. Resumo das User Stories Implementadas

História	Título	Descrição e Responsabilidade Principal
US1 / US1_1	Criação de Empregado	Inclusão de empregados horistas, assalariados e comissionados. Geração de ID único, validações de campos obrigatórios e formato numérico. Persistência em disco testada em US1_1.
US2 / US2_1	Remoção de Empregado	Exclusão de colaboradores cadastrados via ID. Validação de existência do ID informado.
US3 / US3_1	Cartão de Ponto	Lançamento de horas diárias trabalhadas para horistas via classe <code>point_card_pay_u</code> . Consulta de horas normais (até 8h) e horas extras em intervalos de datas [dataInicial, dataFinal).
US4 / US4_1	Lançamento de Venda	Registro de comprovantes de vendas (<code>ResultadoVenda</code>) para comissionados. Consulta de vendas efetuadas em período de datas.
US5 / US5_1	Taxas de Serviço Sindicais	Lançamento de débitos de serviços prestados pelo sindicato via ID de membro sindical (<code>TaxaServico</code>). Consulta agregada de taxas no período.
US6 / US6_1	Alteração de Empregado	Atualização cadastral completa: nome, endereço, troca dinâmica de tipo de contrato com preservação de histórico, dados sindicais e dados de contas bancárias.
US7	Rodar Folha de Pagamento	Cálculo financeiro consolidado dos pagamentos devidos na data informada, apuração de horas, comissões, deduções de taxas sindicais, débitos acumulados e geração de relatório tabular formatado idêntico às especificações oficiais.
US8	Undo / Redo	Capacidade de desfazer e refazer qualquer comando modificador executado desde o início da sessão. Isolamento transacional com rollback automático diante de erros.

4. Documentação das Classes e Métodos

4.1. Classe `Facade`

CAMADA DE FACHADA `br.ufal.ic.p2.wepayu.Facade`

Responsável por expor a interface oficial do sistema para os scripts do EasyAccept. Não contém regras de negócio; carrega a instância serializada de `Sistema` e repassa as chamadas de forma transparente.

```
public void zerarSistema() throws Exception
```

Limpa todos os registros do sistema (empregados, histórico de folhas), permitindo que a ação seja desfeita posteriormente via `undo()`.

```
public String criarEmpregado(String nome, String endereco, String tipo, String salario) throws Exception
```

Cria um novo empregado horista ou assalariado.

Parâmetros: `nome` - Nome completo `endereco` - Endereço residencial `tipo` - "horista" ou "assalariado" `salario` - Valor numérico positivo (por hora ou mensal)

Retorno: Identificador único (UUID) do empregado gerado

Exceções: Validações de campos nulos, vazios, não-numéricos, negativos ou tipo inválido.

```
public String criarEmpregado(String nome, String endereco, String tipo, String salario, String comissao) throws Exception
```

Cria um empregado do tipo comissionado com salário mensal e taxa percentual de comissão.

Parâmetros: `comissao` - Taxa numérica não-negativa sobre as vendas **Retorno:** Identificador único do empregado gerado

```
public String getAtributoEmpregado(String emp, String atributo) throws Exception
```

Recupera o valor formatado de qualquer atributo do empregado (nome, endereco, tipo, salario, comissao, sindicalizado, idSindicato, taxaSindical, metodoPagamento, banco, agencia, contaCorrente).

```
public String getEmpregadoPorNome(String nome, int indice) throws Exception
```

Retorna o ID do n -ésimo empregado cujo nome seja exatamente igual ao fornecido (índice 1-based).

```
public void removerEmpregado(String emp) throws Exception
```

Remove do cadastro o empregado correspondente ao ID informado. Operação reversível por `undo()`.

```
public void lancaCartao(String emp, String data, String horas) throws Exception
```

Registra uma jornada de trabalho no dia para o empregado horista informado.

```
public String getHorasNormaisTrabalhadas(String emp, String dataInicial, String dataFinal) throws Exception
```

Soma as horas normais (até 8h/dia) no intervalo [dataInicial, dataFinal) para o empregado horista.

```
public String getHorasExtrasTrabalhadas(String emp, String dataInicial, String dataFinal) throws Exception
```

Soma as horas extras (excedentes a 8h/dia) no intervalo [dataInicial, dataFinal) para o empregado horista.

```
public void lancaVenda(String emp, String data, String valor) throws Exception
```

Lança o comprovante de uma venda efetuada para um empregado comissionado.

```
public String getVendasRealizadas(String emp, String dataInicial, String dataFinal) throws Exception
```

Soma o total monetário das vendas registradas para o empregado comissionado no intervalo especificado.

```
public void alteraEmpregado(String emp, String atributo, String valor) throws Exception
```

Altera atributos simples (nome, endereco, salario, comissao, sindicalizado=false, metodoPagamento=correios|emMaos) ou tipo para horista/assalariado.

```
public void alteraEmpregado(String emp, String atributo, String valor, String arg4) throws Exception
```

Altera o tipo do empregado para comissionado (informando a comissão em arg4) ou horista (informando o novo salário horário em arg4).

```
public void alteraEmpregado(String emp, String atributo, String valor, String idSindicato, String taxaSindical) throws Exception
```

Altera o status sindical do empregado para true, registrando a identificação sindical única e o valor da taxa periódica.

```
public void alteraEmpregado(String emp, String atributo, String valor, String banco, String agencia, String contaCorrente) throws Exception
```

Altera o método de pagamento para "banco", registrando o nome do banco, número da agência e conta corrente.

```
public void lancaTaxaServico(String membro, String data, String valor) throws Exception
```

Registra uma taxa de serviço cobrada pelo sindicato a partir da matrícula sindical do associado.

```
public String getTaxasServico(String emp, String dataInicial, String dataFinal) throws Exception
```

Soma as taxas de serviço cobradas do empregado no intervalo especificado.

```
public String totalFolha(String data) throws Exception
```

Calcula e retorna o montante total líquido da folha de pagamento na data especificada, sem aplicar descontos definitivos nem alterar o estado dos empregados.

```
public void rodaFolha(String data, String saida) throws Exception
```

Executa o fechamento oficial da folha de pagamento para a data, aplicando as deduções aos empregados, gravando o relatório formatado no arquivo de saída indicado e registrando o estado na pilha de `undo`.

```
public int getNumeroDeEmpregados()
```

Retorna o número total de empregados atualmente cadastrados no sistema.

```
public void undo() throws Exception
```

Desfaz o último comando modificador realizado. Lança `"Nao ha comando a desfazer."` caso a pilha esteja vazia.

```
public void redo() throws Exception
```

Refaz a última operação desfeita. Lança `"Nao ha comando a refazer."` caso não haja comandos para refazer.

```
public void encerrarSistema() throws Exception
```

Marca o sistema como encerrado e salva o estado em disco no arquivo `sistema.dat`. Impede comandos subsequentes na sessão atual.

4.2. Classe `Sistema`

NÚCLEO DE NEGÓCIO `br.ufal.ic.p2.wepayu.Sistema`

Implementa `Serializable`. Contém o mapa central de empregados (`Map<String, Empregado> empregados`), o cache de folhas geradas (`folhaGeradaPorData`) e o controle transacional de snapshots.

Atributos e Gestão de Estado

- `empregados` : `LinkedHashMap` mantendo a ordem de inserção dos empregados.
- `undoStack` : Pilha transitória (`transient Stack<byte[]>`) contendo snapshots dos estados anteriores.
- `redoStack` : Pilha transitória (`transient Stack<byte[]>`) contendo snapshots para refazer.
- `sistemaEncerrado` : Flag booleana transitória que indica se `encerrarSistema()` foi invocado.

Métodos de Controle Transacional Interno

- `executarComandoModificador(ComandoSemRetorno cmd)` : Executa ações que não retornam valor sob garantia transacional. Se ocorrer erro, desfaz alterações em memória via `restaurarEstado`.
- `executarComandoModificadorComRetorno(ComandoComRetorno<T> cmd)` : Versão genérica que executa comandos com retorno (como criação de empregado que devolve o ID).
- `clonarEstado()` : Serializa o objeto `EstadoSistema` em um array de bytes isolado.
- `restaurarEstado(byte[] bytes)` : Deserializa o array de bytes e substitui integralmente os dados em memória.
- `salvar(Sistema s)` e `carregar()` : Persistem e recuperam o estado do sistema no arquivo `sistema.dat`.

4.3. Classe `FolhaDePagamento`

PROCESSAMENTO FINANCEIRO `br.ufal.ic.p2.wepayu.FolhaDePagamento`

Concentra a lógica matemática, financeira e de formatação tabular dos pagamentos. Produz saídas com alinhamento rigoroso de colunas, totalizadores por categoria e total geral.

```
public static ResultadoFolha calcular(Map<String, Empregado> empregados, LocalDate data, boolean aplicarDeduccoes)
```

Percorre todos os empregados do sistema e verifica quem tem direito a pagamento na data especificada:

- **Horistas:** Pagos em sextas-feiras. O período de cálculo compreende desde o sábado anterior até a sexta-feira do pagamento (7 dias). Horas ≤ 8 são pagas pelo salário horário regular; horas > 8 recebem multiplicador de 1.5x (hora extra).
- **Assalariados:** Pagos no último dia útil do mês (se o último dia cair em sábado, paga-se na sexta-feira dia 30; se domingo, na sexta-feira dia 29). Recebem o salário integral mensal contratado.
- **Comissionados:** Pagos em sextas-feiras a cada 2 semanas. Recebem o salário fixo quinzenal $((\text{salarioMensal} \times 12 / 52) \times 2)$ acrescido do percentual de comissão sobre todas as vendas realizadas nas últimas duas semanas.
- **Deduções Sindicais:** Para cada membro sindicalizado, calcula a taxa sindical periódica proporcional somada às taxas de serviço acumuladas no período e eventuais débitos sindicais anteriores. Se `aplicarDeduccoes` for verdadeiro, o débito do empregado é atualizado para o ciclo seguinte.

4.4. Modelo de Domínio (Entidades de Empregado)

MODELOS Pacote `br.ufal.ic.p2.wepayu.models`

`Empregado.java` (Classe Abstrata)

Superclasse de todos os colaboradores. Encapsula:

- Identificação única (`id`), nome completo e endereço.
- Tipo de contrato (`"horista"` , `"assalariado"` , `"comissionado"`).
- Status sindical (`sindicalizado`), matrícula sindical (`idSindicato`), taxa mensal (`taxaSindical`) e lista de taxas de serviço adicionais (`taxasServico`).
- Saldo devedor sindical não quitado em folhas anteriores (`debitoSindicato`).
- Método de pagamento (`metodoPagamento`), banco, agência e conta corrente.

`EmpregadoHorista.java`

Estende `Empregado` . Adiciona:

- `salarioHorario` : Valor em reais por hora de trabalho.
- `cartoes` : Coleção de instâncias de `point_card_pay_u` submetidas pelo colaborador.
- Métodos: `adicionarCartao(point_card_pay_u cartao)` e `getCartoes()` .

`EmpregadoAssalariado.java`

Estende `Empregado` . Adiciona:

- `salarioMensal` : Remuneração fixa mensal estipulada em contrato.

`EmpregadoComissionado.java`

Estende `EmpregadoAssalariado` . Adiciona:

- `comissao` : Taxa percentual (ex: "0,05" para 5%) aplicada sobre as vendas brutas realizadas.
- `vendas` : Coleção de comprovantes de vendas (`ResultadoVenda`).
- Métodos: `adicionarVenda(ResultadoVenda venda)` e `getVendas()` .

4.5. Registros Auxiliares

```
public class point_card_pay_u implements Serializable
```

Representa a folha de ponto diária de um horista.

- `LocalDate getData()` : Data da jornada.
- `double getHoras()` : Total de horas trabalhadas no dia.
- `double getHorasNormais()` : Retorna o total de horas diárias até o limite de 8 horas ($\min(\text{horas}, 8.0)$).
- `double getHorasExtras()` : Retorna as horas excedentes a 8 horas diárias ($\max(0.0, \text{horas} - 8.0)$).

```
public class ResultadoVenda implements Serializable
```

Comprovante de venda realizada por um comissionado. Armazena a data da transação (`LocalDate`) e o valor monetário total (`double`).

```
public class TaxaServico implements Serializable
```

Registro de cobrança de serviço avulso prestado pelo sindicato a um membro filiado. Armazena a data da prestação (`LocalDate`) e o valor do encargo (`double`).

4.6. Tratamento de Exceções

O sistema utiliza exceções checadas e a classe customizada `EmpregadoNaoExisteException` para sinalizar divergências de negócio com exatidão semiótica exigida pelos testes:

- `"Empregado nao existe."` : Lançada quando um ID inexistente é referenciado.
- `"Identificacao do empregado nao pode ser nula."` : Para IDs nulos ou strings vazias.
- `"Empregado nao eh horista."` / `"Empregado nao eh comissionado."` / `"Empregado nao eh sindicalizado."` : Para operações incompatíveis com a categoria do empregado.
- `"Data invalida."` / `"Data inicial nao pode ser posterior aa data final."` : Para inconsistências de calendário ou intervalos temporais inválidos.
- `"Nao pode dar comandos depois de encerrarSistema."` : Proteção contra chamadas pós-fechamento do sistema.
- `"Nao ha comando a desfazer."` / `"Nao ha comando a refazer."` : Para pilhas de histórico vazias.

5. Ciclo de Vida de Execução, Undo/Redo e Persistência

O ciclo de vida do WePayU em uma sessão padrão ocorre conforme o fluxo abaixo:

1. **Inicialização:** Ao instanciar `Facade`, o método `Sistema.carregar()` recupera o arquivo `sistema.dat`. Caso não exista, instancia um novo `Sistema` com coleções vazias.
2. **Execução de Comandos:** A cada comando modificador, um snapshot do estado em memória é empilhado na `undoStack` e a `redoStack` é limpa. Se houver falha de validação, ocorre rollback automático.
3. **Consultas:** Consultas a atributos, horas, vendas, taxas e totalização da folha não afetam as pilhas de histórico.
4. **Desfazer / Refazer:** Comandos `undo()` e `redo()` transferem os snapshots entre as pilhas e substituem os dados em memória sem perda de integridade.
5. **Encerramento:** O comando `encerrarSistema()` bloqueia a aceitação de novos comandos e persiste os dados consolidados no arquivo binário `sistema.dat`.

6. Guia de Compilação e Execução dos Testes

6.1. Compilação do Código Fonte

No terminal PowerShell na raiz do projeto:

```
javac -parameters -cp "lib/easyaccept.jar;src" -d out (Get-ChildItem -Recurse -Filter *.java src | ForEach-Object { $_.FullName })
```

6.2. Execução da Suíte de Testes EasyAccept

```
java -cp "out;lib/easyaccept.jar" Main
```

Resultado Verificado: Todos os 590 testes formais do EasyAccept das User Stories US1 a US8 executam com **100% de sucesso (0 erros, 0 falhas)**.